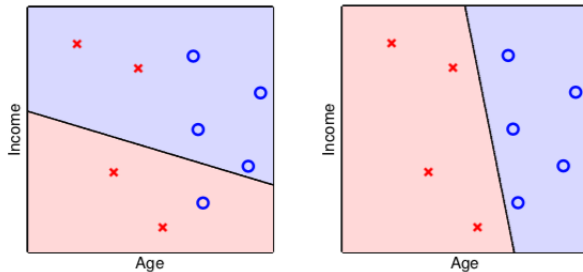




CSCE 633-700: Machine Learning

Linear Perceptron Algorithm

Perceptron: Representation



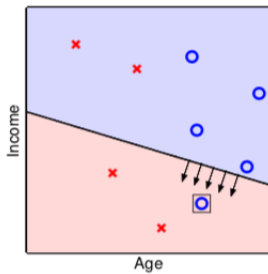
A perceptron fits the data by using a line to separate the $+1$ from -1 data.

Fitting the data: How to find a hyperplane that *separates* the data?

(“It’s obvious - just look at the data and draw the line,” is not a valid solution.)

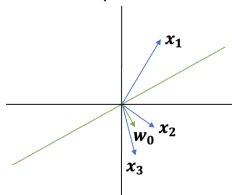
Perceptron Learning Algorithm

Idea! Start with some weight vector and try to improve it.

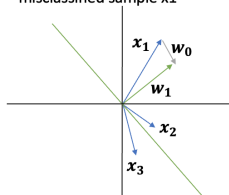


Perceptron Learning Algorithm: Example

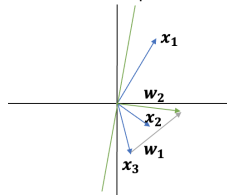
a) Random initialization of decision boundary



b) After correction with misclassified sample x_1



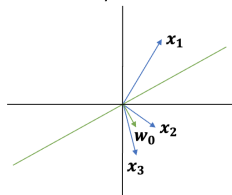
c) After correction with misclassified sample x_3



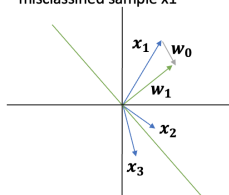
Assuming that $\{x_1, x_2, x_3\}$ belong to the same class ($y = 1$), after the initial updates, successive corrections become smaller and the algorithm “fine tunes” the position of the weight vector.

Perceptron Learning Algorithm: Example

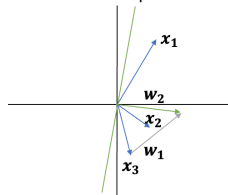
a) Random initialization of decision boundary



b) After correction with misclassified sample x_1



c) After correction with misclassified sample x_3

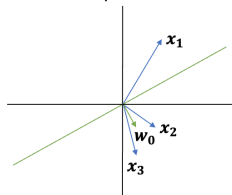


a) $w_0^T x_1 < 0$, $w_0^T x_2 > 0$, $w_0^T x_3 > 0$: Sample 1 is incorrectly classified, therefore we need to adjust w_0 so that we correct this.

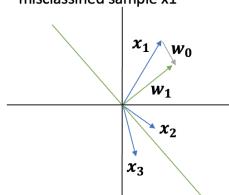
b) We do so by adding x_1 to w_0 , since this will shift the line so that x_1 lies on the right side of the line:
 $w_1 = w_0 + x_1$

Perceptron Learning Algorithm: Example

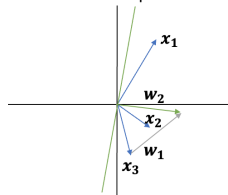
a) Random initialization of decision boundary



b) After correction with misclassified sample x1



c) After correction with misclassified sample x3

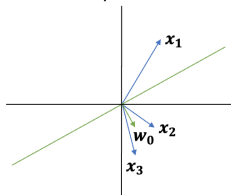


b) $\mathbf{w}_1^T \mathbf{x}_1 > 0$, $\mathbf{w}_1^T \mathbf{x}_2 > 0$, $\mathbf{w}_1^T \mathbf{x}_3 < 0$: Sample 3 is incorrectly classified, therefore we need to adjust $\mathbf{w}_1$ so that we correct this.

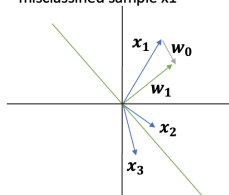
c) We do so by adding $\mathbf{x}_3$ to $\mathbf{w}_1$, since this will shift the line so that $\mathbf{x}_3$ lies on the right side of the line:
 $\mathbf{w}_2 = \mathbf{w}_1 + \mathbf{x}_3$

Perceptron Learning Algorithm: Example

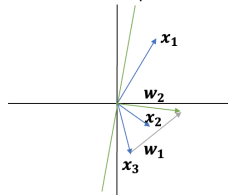
a) Random initialization of decision boundary



b) After correction with misclassified sample x_1



c) After correction with misclassified sample x_3



c) $w_2^T x_1 > 0$, $w_2^T x_2 > 0$, $w_2^T x_3 > 0$: All samples are correctly classified. No further action is needed.

Perceptron Learning Algorithm

A simple iterative method

Incremental learning on single example at a time

- ① Initialization $\mathbf{w}(\mathbf{0}) = \mathbf{0}$ (or any other vector)
- ② for $t = 1, 2, 3, \dots$
 - a From $\{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_N, y_N)\}$ pick a misclassified sample
 - b Call the misclassified sample $(\mathbf{x}_s, y_s)$: $\text{sign}(\mathbf{w}(\mathbf{t})^T \mathbf{x}_s) \neq y_s$
 $(\mathbf{w}(\mathbf{t})^T \mathbf{x}_s = -1 \text{ if } y_s = 1; \mathbf{w}(\mathbf{t})^T \mathbf{x}_s = 1 \text{ if } y_s = -1)$
 - c Update the weight:
 $\mathbf{w}(\mathbf{t} + 1) = \mathbf{w}(\mathbf{t}) + y_s \mathbf{x}_s$
 - d $t \leftarrow t + 1$

Perceptron Learning Algorithm

Algorithm Learning Rate

- Vectors $\mathbf{w}$ are not necessarily normalized
- If $\|\mathbf{w}(t)\| \gg \|\mathbf{x}_s\|$ the new weight vector $\mathbf{w}(t) + \mathbf{x}_s$ is almost equal to $\mathbf{w}(t)$
- We can add a **learning rate** $\alpha > 0$ in the update
 - $\mathbf{w}(t + 1) = \mathbf{w}(t) \pm \alpha \mathbf{x}_s$
 - if $\alpha \gg 0$, $\mathbf{x}_s$ will have a large impact on the update
 - if $\alpha \equiv 0$, $\mathbf{x}_s$ will not influence much the update
 - Learning rate a typical **hyperparameter** of many learning algorithms and depends on the data of each problem
 - It can be larger in the first learning steps, and decreasing later on, e.g., $\alpha = \frac{1}{t}c$, where t is the iteration and c is a constant

Perceptron Learning Algorithm

Algorithm Convergence

- The rule update considers a training sample at a time and may “destroy” the classification of other samples
- If the data can be fit by a **linear separator** (**linearly separable**), then after some finite number of steps, PLA is guaranteed to arrive to a correct solution.
- What if the data cannot be fit by a perceptron?
 - We can find infinitely many combinations of perceptrons that fit the data → neural networks

Perceptron Learning Algorithm

Algorithm Cost

- PLA is a local, greedy algorithm
- This can lead to an exponential number of updates of the weight
- In the following figure:
 - Two almost antiparallel vectors are to be classified in the same half-space
 - PLA rotates the separation line in one of the two directions and will require more and more time when the angle between the two vectors approaches 180 degrees

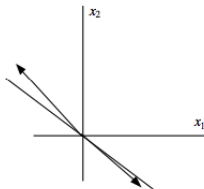


Fig. 4.13. Worst case for perceptron learning (input space)

Parametric v.s. non-parametric models

- Many possible ways to categorize learning models
- One way is to ask this question
 - Does the model have a fixed number of parameters or does it grow with the amount of training data?
- Non-parametric models (or instance/memory-based)
 - more flexible
 - computationally intractable for large datasets
 - e.g. K-NN
- Parametric models
 - faster to use
 - make strong assumptions about data
 - e.g. linear perceptron, linear regression